Go from Data to Strategy: Tepper School of Business

Using Learning Rate Schedule in PyTorch Training

by **Adrian Tam** on April 8, 2023 in **Deep Learning with PyTorch** 8

Share Share Post

Training a neural network or large deep learning model is a difficult optimization task.

The classical algorithm to train neural networks is called stochastic gradient descent. It has been well established that you can achieve increased performance and faster training on some problems by using a learning rate that changes during training.

In this post, you will discover what is learning rate schedule and how you can use different learning rate schedules for your neural network models in PyTorch.

After reading this post, you will know:

- The role of learning rate schedule in model training
- How to use learning rate schedule in PyTorch training loop
- How to set up your own learning rate schedule

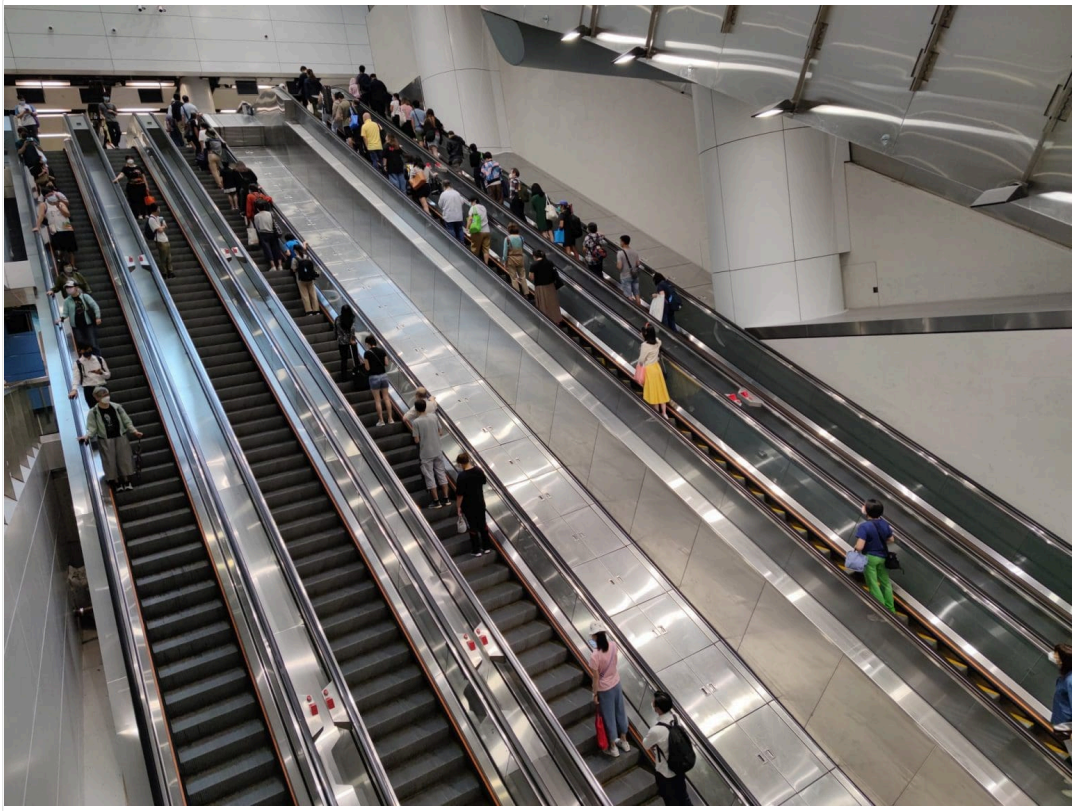
Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Let's get started.



Using Learning Rate Schedule in PyTorch Training
Photo by Cheung Yin. Some rights reserved.

Overview

This post is divided into three parts; they are

- Learning Rate Schedule for Training Models
- Applying Learning Rate Schedule in PyTorch Training
- Custom Learning Rate Schedules

Learning Rate Schedule for Training Models

Gradient descent is an algorithm of numerical optimization. What it does is to update parameters using the formula:

$$w := w - \alpha \frac{dy}{dw}$$

In this formula, w is the parameter, e.g., the weight in a neural network, and y is the objective, e.g., the loss function. What it does is to move w to the direction that you can minimize y. The direction is provided by the differentiation, $\frac{dy}{dw}$, but how much you should move w is controlled by the **learning rate** α.

An easy start is to use a constant learning rate in gradient descent algorithm. But you can do better with a **learning rate schedule**. A schedule is to make learning rate adaptive to the gradient descent optimization procedure, so you can increase performance and reduce training time.

In the neural network training process, data is feed into the network in batches, with many batches in one epoch. Each batch triggers one training step, which the gradient descent algorithm updates the parameters once. However, usually the learning rate schedule is updated once for each training epoch only.

You can update the learning rate as frequent as each step but usually it is updated once per epoch because you want to know how the network performs in order to determine how the learning rate should update. Regularly, a model is evaluated with validation dataset once per epoch.

There are multiple ways of making learning rate adaptive. At the beginning of training, you may prefer a larger learning rate so you improve the network coarsely to speed up the progress. In a very complex neural network model, you may also prefer to gradually increasse the learning rate at the beginning because you need the network to explore on the different dimensions of prediction. At the end of training, however, you always want to have the learning rate smaller. Since at that time, you are about to get the best performance from the model and it is easy to overshoot if the learning rate is large.

Therefore, the simplest and perhaps most used adaptation of the learning rate during training are techniques that reduce the learning rate over time. These have the benefit of making large changes at the beginning of the training procedure when larger learning rate values are used and decreasing the learning rate so that a smaller rate and, therefore, smaller training updates are made to weights later in the training procedure.

This has the effect of quickly learning good weights early and fine-tuning them later.

Next, let's look at how you can set up learning rate schedules in PyTorch.

Kick-start your project with my book Deep Learning with PyTorch. It provides **self-study tutorials** with **working code**.

Applying Learning Rate Schedules in PyTorch Training

In PyTorch, a model is updated by an optimizer and learning rate is a parameter of the optimizer. Learning rate schedule is an algorithm to update the learning rate in an optimizer.

Below is an example of creating a learning rate schedule:

```
import torch
import torch.optim as optim
import torch.optim.lr_scheduler as lr_scheduler

scheduler = lr_scheduler.LinearLR(optimizer, start_factor=1.0, end_factor=0.3, total_iters=10)
```

There are many learning rate scheduler provided by PyTorch in `torch.optim.lr_scheduler` submodule. All the scheduler needs the optimizer to update as first argument. Depends on the scheduler, you may need to provide more arguments to set up one.

Let's start with an example model. In below, a model is to solve the ionosphere binary classification problem. This is a small dataset that you can download from the UCI Machine Learning repository. Place the data file in your working directory with the filename `ionosphere.csv`.

The ionosphere dataset is good for practicing with neural networks because all the input values are small numerical values of the same scale.

A small neural network model is constructed with a single hidden layer with 34 neurons, using the ReLU activation function. The output layer has a single neuron and uses the sigmoid activation function in order to output probability-like values.

Plain stochastic gradient descent algorithm is used, with a fixed learning rate 0.1. The model is trained for 50 epochs. The state parameters of an optimizer can be found in `optimizer.param_groups`; which the learning rate is a floating point value at `optimizer.param_groups[0]["lr"]`. At the end of each epoch, the learning rate from the optimizer is printed.

The complete example is listed below.

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# load dataset, split into input (X) and output (y) variables
dataframe = pd.read_csv("ionosphere.csv", header=None)
dataset = dataframe.values
X = dataset[:,0:34].astype(float)
y = dataset[:,34]

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# create model
model = nn.Sequential(
    nn.Linear(34, 34),
    nn.ReLU(),
    nn.Linear(34, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 50
batch_size = 24
batch_start = torch.arange(0, len(X_train), batch_size)
lr = 0.1
loss_fn = nn.BCELoss()
```

```
optimizer = optim.SGD(model.parameters(), lr=lr)
model.train()
for epoch in range(n_epochs):
    for start in batch_start:
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print("Epoch %d: SGD lr=%.4f" % (epoch, optimizer.param_groups[0]["lr"]))

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

Running this model produces:

```
Epoch 0: SGD lr=0.1000
Epoch 1: SGD lr=0.1000
Epoch 2: SGD lr=0.1000
Epoch 3: SGD lr=0.1000
Epoch 4: SGD lr=0.1000
...
Epoch 45: SGD lr=0.1000
Epoch 46: SGD lr=0.1000
Epoch 47: SGD lr=0.1000
Epoch 48: SGD lr=0.1000
Epoch 49: SGD lr=0.1000
Model accuracy: 86.79%
```

You can confirm that the learning rate didn't change over the entire training process. Let's make the training process start with a larger learning rate and end with a smaller rate. To introduce a learning rate scheduler, you need to run its `step()` function in the training loop. The code above is modified into the following:

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import torch.optim.lr_scheduler as lr_scheduler
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# load dataset, split into input (X) and output (y) variables
dataframe = pd.read_csv("ionosphere.csv", header=None)
dataset = dataframe.values
X = dataset[:,0:34].astype(float)
y = dataset[:,34]

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# create model
model = nn.Sequential(
    nn.Linear(34, 34),
    nn.ReLU(),
    nn.Linear(34, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 50
batch_size = 24
batch_start = torch.arange(0, len(X_train), batch_size)
lr = 0.1
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=lr)
scheduler = lr_scheduler.LinearLR(optimizer, start_factor=1.0, end_factor=0.5, total_iters=30)
model.train()
for epoch in range(n_epochs):
    for start in batch_start:
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    before_lr = optimizer.param_groups[0]["lr"]
    scheduler.step()
    after_lr = optimizer.param_groups[0]["lr"]
```

```
    print("Epoch %d: SGD lr %.4f -> %.4f" % (epoch, before_lr, after_lr))

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

It prints:

```
Epoch 0: SGD lr 0.1000 -> 0.0983
Epoch 1: SGD lr 0.0983 -> 0.0967
Epoch 2: SGD lr 0.0967 -> 0.0950
Epoch 3: SGD lr 0.0950 -> 0.0933
Epoch 4: SGD lr 0.0933 -> 0.0917
...
Epoch 28: SGD lr 0.0533 -> 0.0517
Epoch 29: SGD lr 0.0517 -> 0.0500
Epoch 30: SGD lr 0.0500 -> 0.0500
Epoch 31: SGD lr 0.0500 -> 0.0500
...
Epoch 48: SGD lr 0.0500 -> 0.0500
Epoch 49: SGD lr 0.0500 -> 0.0500
Model accuracy: 88.68%
```

In the above, `LinearLR()` is used. It is a linear rate scheduler and it takes three additional parameters, the `start_factor`, `end_factor`, and `total_iters`. You set `start_factor` to 1.0, `end_factor` to 0.5, and `total_iters` to 30, therefore it will make a multiplicative factor decrease from 1.0 to 0.5, in 10 equal steps. After 10 steps, the factor will stay at 0.5. This factor is then multiplied to the original learning rate at the optimizer. Hence you will see the learning rate decreased from $0.1 \times 1.0 = 0.1$ to $0.1 \times 0.5 = 0.05$.

Besides `LinearLR()`, you can also use `ExponentialLR()`, its syntax is:

```
scheduler = lr_scheduler.ExponentialLR(optimizer, gamma=0.99)
```

If you replaced `LinearLR()` with this, you will see the learning rate updated as follows:

```
Epoch 0: SGD lr 0.1000 -> 0.0990
Epoch 1: SGD lr 0.0990 -> 0.0980
Epoch 2: SGD lr 0.0980 -> 0.0970
Epoch 3: SGD lr 0.0970 -> 0.0961
Epoch 4: SGD lr 0.0961 -> 0.0951
...
Epoch 45: SGD lr 0.0636 -> 0.0630
Epoch 46: SGD lr 0.0630 -> 0.0624
Epoch 47: SGD lr 0.0624 -> 0.0617
Epoch 48: SGD lr 0.0617 -> 0.0611
Epoch 49: SGD lr 0.0611 -> 0.0605
```

In which the learning rate is updated by multiplying with a constant factor `gamma` in each scheduler update.

Custom Learning Rate Schedules

There is no general rule that a particular learning rate schedule works the best. Sometimes, you like to have a special learning rate schedule that PyTorch didn't provide. A custom learning rate schedule can be defined using a custom function. For example, you want to have a learning rate that:

$$lr_n = \frac{lr_0}{1 + \alpha n}$$

on epoch n, which lr_0 is the initial learning rate, at epoch 0, and α is a constant. You can implement a function that given the epoch n calculate learning rate lr_n:

```
def lr_lambda(epoch):
    # LR to be 0.1 * (1/1+0.01*epoch)
    base_lr = 0.1
    factor = 0.01
    return base_lr/(1+factor*epoch)
```

Then, you can set up a `LambdaLR()` to update the learning rate according to this function:

```
scheduler = lr_scheduler.LambdaLR(optimizer, lr_lambda)
```

Modifying the previous example to use `LambdaLR()`, you have the following:

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import torch.optim.lr_scheduler as lr_scheduler
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

```

```
# load dataset, split into input (X) and output (y) variables
dataframe = pd.read_csv("ionosphere.csv", header=None)
dataset = dataframe.values
X = dataset[:,0:34].astype(float)
y = dataset[:,34]

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# create model
model = nn.Sequential(
    nn.Linear(34, 34),
    nn.ReLU(),
    nn.Linear(34, 1),
    nn.Sigmoid()
)

def lr_lambda(epoch):
    # LR to be 0.1 * (1/1+0.01*epoch)
    base_lr = 0.1
    factor = 0.01
    return base_lr/(1+factor*epoch)

# Train the model
n_epochs = 50
batch_size = 24
batch_start = torch.arange(0, len(X_train), batch_size)
lr = 0.1
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=lr)
scheduler = lr_scheduler.LambdaLR(optimizer, lr_lambda)
model.train()
for epoch in range(n_epochs):
    for start in batch_start:
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    before_lr = optimizer.param_groups[0]["lr"]
    scheduler.step()
    after_lr = optimizer.param_groups[0]["lr"]
    print("Epoch %d: SGD lr %.4f -> %.4f" % (epoch, before_lr, after_lr))

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

Which produces:

```
Epoch 0: SGD lr 0.0100 -> 0.0099
Epoch 1: SGD lr 0.0099 -> 0.0098
Epoch 2: SGD lr 0.0098 -> 0.0097
Epoch 3: SGD lr 0.0097 -> 0.0096
Epoch 4: SGD lr 0.0096 -> 0.0095
...
Epoch 45: SGD lr 0.0069 -> 0.0068
Epoch 46: SGD lr 0.0068 -> 0.0068
Epoch 47: SGD lr 0.0068 -> 0.0068
Epoch 48: SGD lr 0.0068 -> 0.0067
Epoch 49: SGD lr 0.0067 -> 0.0067
```

Note that although the function provided to `LambdaLR()` assumes an argument `epoch`, it is not tied to the epoch in the training loop but simply counts how many times you invoked `scheduler.step()`.

Tips for Using Learning Rate Schedules

This section lists some tips and tricks to consider when using learning rate schedules with neural networks.

- **Increase the initial learning rate**. Because the learning rate will very likely decrease, start with a larger value to decrease from. A larger learning rate will result in a lot larger changes to the weights, at least in the beginning, allowing you to benefit from the fine-tuning later.
- **Use a large momentum**. Many optimizers can consider momentum. Using a larger momentum value will help the optimization algorithm continue to make updates in the right direction when your learning rate shrinks to small values.
- **Experiment with different schedules**. It will not be clear which learning rate schedule to use, so try a few with different configuration options and see what works best on your problem. Also, try schedules that change exponentially and even schedules that respond to the accuracy of

your model on the training or test datasets.

Further Readings

Below is the documentation for more details on using learning rates in PyTorch:

- How to adjust learning rate, from PyTorch documentation

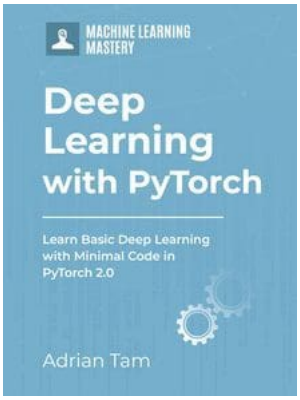
Summary

In this post, you discovered learning rate schedules for training neural network models.

After reading this post, you learned:

- How learning rate affects your model training
- How to set up learning rate schedule in PyTorch
- How to create a custom learning rate schedule

Get Started on Deep Learning with PyTorch!



Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation*, *training*, *evaluation*, *hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises

SEE WHAT'S INSIDE

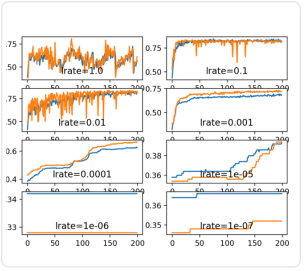
Share Share Post

More On This Topic

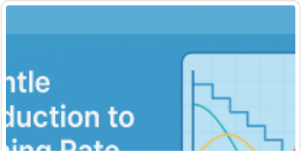
How to Configure the Learning Rate When Training…

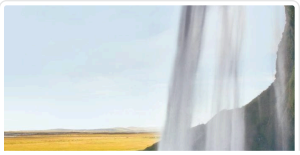
Using Learning Rate Schedules for Deep Learning…

Tune Learning Rate for Gradient Boosting with…



Understand the Impact of Learning Rate on Neural…





About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
View all posts by Adrian Tam →

‹ Using Dropout Regularization in PyTorch Models
Training a PyTorch Model with DataLoader and Dataset ›

8 Responses to *Using Learning Rate Schedule in PyTorch Training*

Tanner July 27, 2023 at 11:56 pm #

REPLY

A fine article, but full of grammatical errors that sometimes make it hard to follow.

James Carmichael July 28, 2023 at 9:08 am #

REPLY

Thank you Tanner for your feedback!

FelixHao December 19, 2023 at 8:22 pm #

REPLY

lr_lambda doesn't need to multiply base_lr at the end. If you look at the PyTorch's source code, LambdaLR.get_lr does that for you.

James Carmichael December 20, 2023 at 9:43 am #

REPLY

Thank you for the clarification FelixHao!

SuBele February 9, 2024 at 2:47 pm #

REPLY

Can this learning rate scheduler be also used with Adam?

James Carmichael February 11, 2024 at 12:58 am #

REPLY

Hi SuBele…This would be unecessary when using Adam.

https://ai.stackexchange.com/questions/35041/do-learning-rate-schedulers-conflict-with-or-prevent-convergence-of-the-adam-opt#:~:text=Because%20Adam%20manages%20learning%20rates,causing%20model%20convergence%20to%20worsen.

Benjamin May 19, 2025 at 7:13 pm #

REPLY

Regarding the 'total_iter' parameter in LinearLR, you say (quoted from the text):

You set start_factor to 1.0, end_factor to 0.5, and total_iters to 30, therefore it will make a multiplicative factor decrease from 1.0 to 0.5, in 10 equal steps. After 10 steps, the factor will stay at 0.5.

I think it should be:

You set start_factor to 1.0, end_factor to 0.5, and total_iters to 30, therefore it will make a multiplicative factor decrease from 1.0 to 0.5, in 30 equal steps. After 30 steps, the factor will stay at 0.5.

Is that correct?

James Carmichael May 20, 2025 at 2:06 am #

REPLY

Yes, you are absolutely correct — the quote in the text is incorrect about the number of steps.

Here's the accurate explanation:

When using `torch.optim.lr_scheduler.LinearLR`, the learning rate is linearly interpolated between `start_factor` and `end_factor` over the number of steps specified by `total_iters`. After those steps, the learning rate stays constant at `end_factor`.

So if you set:

* start_factor = 1.0
* end_factor = 0.5
* total_iters = 30

Then PyTorch will decrease the learning rate factor gradually from 1.0 to 0.5 over 30 steps—not 10. After step 30, the factor will remain fixed at 0.5.

Summary:

* Correct: The learning rate decreases over 30 steps
* Incorrect: Saying it decreases over 10 steps

Thanks for pointing that out—your correction is exactly right.

Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT

Welcome!
I'm *Jason Brownlee* PhD
and I **help developers** get results with **machine learning**.
Read more

Never miss a tutorial:

Picked for you:


PyTorch Tutorial: How to Develop Deep Learning Models with Python


LSTM for Time Series Prediction in PyTorch


Deep Learning with PyTorch (9-Day Mini-Course)


Calculating Derivatives in PyTorch


Develop Your First Neural Network with PyTorch, Step by Step

Loving the Tutorials?

The Deep Learning with PyTorch EBook
is where you'll find the ***Really Good*** stuff.

>> SEE WHAT'S INSIDE

Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. Visit our corporate website to learn more about our mission and team.

PRIVACY | DISCLAIMER | TERMS | CONTACT | SITEMAP | ADVERTISE WITH US

© 2026 Guiding Tech Media All Rights Reserved